An Edge–Cloud Collaborative Framework for Latency Optimization in Prompt Injection Defense

Minyang Shen
College of Computer Science and Technology
Zhejiang University of Technology
Hangzhou, China
221125120306@zjut.edu.cn

Xinggang Fan*
Zhijiang College
Zhejiang University of Technology
Shaoxing, China
xgfan@zjut.edu.cn

Haodong Zou
School of Computer Science and Technology
Anhui University
Hefei, China
zou_hd@163.com

Ruihao Wang
Zhijiang College
Zhejiang University of Technology
Shaoxing, China
202402010119@zzjc.edu.cn

Haiyang Huang*
School of AI and Computer Science
Nantong University
Nantong, China
younghuangh@163.com

2026 International Conference on Sensor-Cloud and Edge Computing System (SCECS) | 979-8-3195-1983-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/SCECS69812.2026.11566062

***Abstract*—Prompt injection is a major input-side risk in Large Language Model (LLM) services. In edge–cloud deployments, a natural design is to let the edge perform lightweight detection and send hard-to-judge requests to the cloud for stronger checking. However, when many such requests are forwarded, cloud-side checking itself can become a queueing bottleneck, substantially increasing end-to-end delay, especially tail latency under heavy load. This paper studies how edge-side risk stratification can be used to decide which requests should receive further cloud-side checking, without scaling the provisioned cloud checking tier. We deploy a lightweight edge detector to divide requests into low-, medium-, and high-risk groups. High-risk requests are blocked at the edge, low-risk requests proceed directly, and medium-risk requests are selectively sent to the cloud or downgraded to a restricted execution mode according to both request risk and congestion state. We formulate this process as an online decision problem and show that it is NP-hard. Based on this formulation, we propose Q-TRiG, a low-overhead queue-aware gating policy for medium-risk traffic. Discrete-event simulations show that Q-TRiG reduces cloud queueing delay, suppresses 99th percentile latency spikes, and enlarges the stable operating region under heavy load. The results suggest that risk-stratified selective cloud checking is an effective way to reduce delay in prompt-injection detection pipelines.**

***Index Terms*—Prompt Injection, Edge–Cloud Collaboration, Queue-aware Gating, Tail Latency, Large Language Models**

I. Introduction

Large Language Models (LLMs) are being widely integrated into online systems such as retrieval-augmented question answering, customer service, code generation, and enterprise workflows [1], [2]. As LLMs transition from offline capability demonstrations to production-grade deployments, input and context governance has increasingly become a critical component of system security. Among input-side threats, prompt injection is especially concerning because adversarial instructions can be embedded into user prompts or external content, causing the model to ignore system intent or execute unintended actions. Recent studies indicate that prompt injections can not only be manually crafted but also automatically generated as highly transferable universal attacks, maintaining robust attack success rates across various defense setups. Consequently, static rule filtering and empirical blacklists struggle to remain effective in the long term [3]–[6]. Furthermore, with the rise of agentic applications involving tool invocation and external content integration, the attack surface of indirect prompt injections in real-world systems has further expanded. This has prompted the research community to systematically evaluate attacks and defenses using benchmark environments that more closely approximate real-world tasks [7]–[10].

A common engineering response is to apply stronger checking to suspicious requests, often in the cloud, where more capable detectors or multi-stage checking pipelines can be deployed. While intuitive, this design introduces an often-overlooked system-level problem: in online services, cloud-side checking is not free. It consumes computation, creates queues under high concurrency, and can become a dominant source of end-to-end delay. As the load rises, forwarding too many requests for stronger checking may sharply increase queueing delay and trigger severe p95/p99 latency degradation. Moreover, increasing cloud-side detection complexity or verification frequency to improve security further reduces the number of requests processable per unit time, exacerbating queueing effects and creating tension between security and Quality of Service (QoS). Although existing work continues to improve prompt injection detectors—emphasizing deployability and low false-positive evaluations [11]–[13], as well as proposing lightweight detection approaches that combine embeddings with traditional classifiers to reduce inference cost [14]—there remains a lack of clear modeling and reproducible system-level evidence on how detector outputs should be coupled with online routing decisions to control tail latency under heavy load and expand the stable operating regime.

From a systems perspective, this paper proposes an edge–

*Corresponding author: Xinggang Fan and Haiyang Huang.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:44:13 UTC from IEEE Xplore. Restrictions apply.

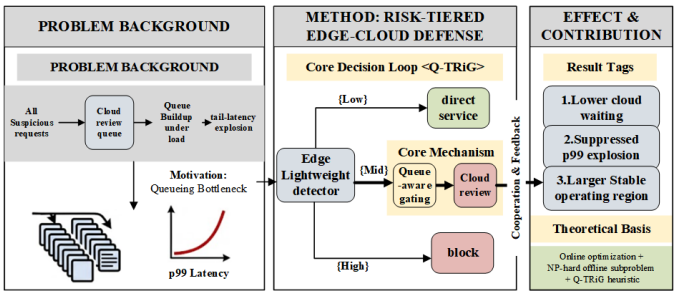


Fig. 1. Proposed edge–cloud collaborative prompt injection defense: Lightweight edge detector classifies requests into low/medium/high risk. Low-risk bypass cloud recheck to service pipeline, high-risk blocked at edge, medium-risk adaptively routed between cloud-side strong recheck and restricted execution via queue-aware gating.

cloud collaborative defense framework, illustrated in Fig. 1, where lightweight edge-side detectors perform three-level risk stratification and a fixed cloud-side strong verification pool handles selected medium-risk requests. Based on this architecture, we formulate routing as an online security-constrained decision problem and develop Q-TRiG, a low-overhead queue-aware gating algorithm, to control cloud queueing delay and tail-latency inflation under heavy loads.

The main contributions of this paper are as follows:

1) We identify a system-level problem in prompt-injection detection for edge–cloud services: forwarding hard-to-judge requests for stronger cloud-side checking can itself cause severe queueing delay under high concurrency. We thus propose a risk-stratified edge–cloud framework for selective cloud-side checking.
2) We formulate this process as an online queue-aware decision problem and show that it is NP-hard. Based on this formulation, we propose Q-TRiG, an interpretable, low-overhead, queue-aware three-stage gating algorithm.
3) We construct an end-to-end queueing simulation framework to quantify cloud waiting times, tail latencies, and stable operating regimes among various strategies under heavy loads, providing reproducible evidence for the benefits of risk-stratified selective cloud checking.

II. Related Work and Motivation

A. Prompt Injection Attacks and Evaluation

Prompt injection attacks induce large language models (LLMs) to deviate from system objectives and execute the attacker's intent by embedding adversarial instructions within input prompts or external content [1], [15]. Recent research has advanced this issue from a small number of manually crafted examples to scalable and transferable automated attacks: Liu et al. [3] proposed a gradient-driven automated framework capable of generating universal injection samples, revealing that the robustness of existing defenses might be overestimated under automated stress testing. Meanwhile, as LLM agents integrate with tools and third-party content, indirect prompt injection has emerged as a threat more closely aligned with real-world deployments, driving the research community to construct more complex evaluation environments [7]. The latest work, AgentDyn, further points out that existing benchmarks are deficient in dynamic open-ended tasks and handling "useful but untrusted content," reporting that security and over-defense issues with current defenses remain prominent under more realistic settings [16], [17].

The aforementioned works primarily emphasize attack feasibility and evaluation benchmarks [18], but there is a relative lack of system-level evidence regarding "how defense strategies maintain tail latency and stable operating regimes during high concurrency" in the context of online services. Particularly in tool-integrated agent scenarios, indirect prompt injections trigger more complex risks of data leakage and unauthorized access via external content, urgently requiring the evaluation of defense strategies' system impacts on a more fine-grained set of test cases. This paper fills this gap from a queueing system perspective, focusing on characterizing the tail latency behavior of different edge-side strategies within the critical load regime.

B. Detection and System-Level Defense

Compared to rule-based or generative self-checking approaches, discriminative detectors are widely adopted due to their controllable inference overhead and ease of deployment. Ayub [19] investigated a scheme that maps prompts into a vector space using text embeddings and combines them with traditional machine learning classifiers for discrimination, which is suitable as a lightweight frontline defense. Furthermore, Jacob et al. [20] pointed out that many existing detectors still exhibit suboptimal deployment metrics, subsequently constructing a benchmark oriented towards deployable detectors and proposing stronger detection schemes. In the context of indirect injection, Chen et al. [21] systematically analyzed the feasibility of "detecting and removing" indirect injections and constructed corresponding evaluation datasets, illustrating that defenses must confront more complex external context injections.

Existing detections and benchmarks predominantly focus on detection performance as the primary objective, rarely coupling detection outputs with the resource constraints of online systems. In particular, there is a lack of system-level quantification of tail latency and queueing delays under a fixed cloud computing budget [22]. This paper explicitly maps the detector outputs into a three-stage risk signal. Using a fixed cloud service pool and a shortest-queue baseline as the backend, we quantify the impact of edge-side risk stratification on tail latency, thereby unifying deployable detection and available Quality of Service (QoS) within the same evaluation framework.

III. Problem Formulation and Queue-Aware Risk Decision

A. System Model and Risk Partitioning

We consider an edge–cloud detection pipeline for LLM services. At time slot t, requests arrive according to a Poisson process with arrival rate λ (req/s), denoted as $\mathcal{Q}_t = \{1, \cdots, Q\}$.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:44:13 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EDGE DETECTORS AND RISK-SPLIT STATISTICS

Embedding Model	Classifier	Accuracy	Risk Split		
			Low-risk	Mid-risk	High-risk
MiniLM	RF	0.9533	56.69%	30.49%	12.82%
OpenAI	RF	0.9723	79.63%	15.58%	4.79%

Let the textual input of request $i \in \mathcal{Q}_t$ be x_i, and let its arrival time be a_i. The edge detector outputs a risk score s_i for x_i, which is mapped into a three-level risk label by two thresholds τ_L and τ_H:

$$r_i = \begin{cases} \text{L}, & s_i < \tau_L, \\ \text{M}, & \tau_L \le s_i < \tau_H, \\ \text{H}, & s_i \ge \tau_H. \end{cases} \tag{1}$$

This three-level design is motivated by the observation that edge-side screening is typically more decisive for clearly benign and clearly suspicious requests, while requests in the middle region carry higher uncertainty. Rather than treating all non-benign requests in the same way, we explicitly isolate this uncertain region and make it the target of adaptive system control. Under the current threshold setting, the edge-side detector adopts a lightweight embedding-plus-classifier pipeline (MiniLM embeddings with a random forest classifier), yielding the following three-stage risk distribution:

$$P_{\text{low}} \approx 0.5669, \quad P_{\text{mid}} \approx 0.3049, \quad P_{\text{high}} \approx 0.1282.$$

Table I summarizes the performance of different detectors and the corresponding risk-split ratios on the dataset. The edge detector acts as a risk-stratification sensor that drives edge-side dispatching and changes the effective arrival pattern into the cloud.DeBERTa [23] is used as the cloud-side strong rechecking model, and its output distribution is used for rechecking decisions. Therefore, its risk proportions are not directly compared with those of the lightweight edge model.

B. End-to-End Latency Model and Decision Variables

After edge detection, the system assigns one of three actions to each request: `EDGE`, meaning no cloud-side rechecking; `CLOUD`, meaning cloud-side rechecking is performed; and `BLOCK`, meaning the request is blocked as high-risk.

Let $u_i \in \{EDGE, CLOUD, BLOCK\}$ denote the action assigned to request x_i. The end-to-end latency $L(u_i)$ is modeled as

$$L(u_i) = \begin{cases} T_e + T_{\text{llm}}, & u_i = \text{EDGE}, \\ T_e + T_n + W_q + T_c + n_i \cdot T_{\text{llm}}, & u_i = \text{CLOUD}, \\ T_e + T_b, & u_i = \text{BLOCK}, \end{cases} \tag{2}$$

where T_e is the edge detection time, T_{llm} is the LLM processing time, T_n is the network delay, $n_i \in \{0, 1\}$ denotes the cloud-side detection outcome, with $n_i = 0$ indicating that x_i is classified as malicious and $n_i = 1$ otherwise, T_c is the cloud-side rechecking service time, W_q is the cloud queueing delay before cloud-side checking, and T_b is the blocking response time. This paper focuses on the queueing effect in the rechecking path. In experiments, T_e is treated as a common constant across strategies and omitted, while T_n and T_c are modeled by fixed means so that only the differences in W_q and tail latency are compared.

The cloud rechecking layer is abstracted as a service pool of k homogeneous detector instances with deterministic service time, using Join-the-Shortest-Queue (JSQ) routing. Let $\mu = \frac{1}{T_c}$ denote the service rate of a single instance. If a fraction p_{send} of requests is sent to the cloud for rechecking, then the effective cloud arrival rate is $\lambda_{cloud} = \lambda p_{send}$, and the cloud utilization is defined as

$$\rho = \frac{\lambda_{cloud}}{k\mu} = \frac{\lambda p_{send}}{k\mu}. \tag{3}$$

When $\rho \to 1$, queueing delay becomes highly sensitive to load. When $\rho > 1$, the system enters an unstable region where queue length and waiting time diverge over time.

C. Problem Definition: Minimizing Mean and Tail Latency Under Security Constraints

Our goal is not to redesign the detector, but to determine how its risk outputs should be used online. Since low-risk requests are handled directly and high-risk requests are blocked, the key decision lies in whether medium-risk requests should still be sent to the cloud for stronger checking.

Let π denote the online decision policy under a fixed cloud service pool configuration. The policy maps each request's risk level and current queue state to an action, i.e., $u_i = \pi(r_i, \mathbf{q}_i)$ with the objective of optimizing system-level latency while satisfying security constraints. Here, $\mathbf{q}_i = \{free_j\}_{j=1}^{k}$ denotes the next available time of each cloud instance when the i-th request arrives, from which the predicted waiting time $\hat{W}$ can be computed. We adopt a mean–tail weighted objective and explicitly model the loss of security margin caused by downgrading medium-risk requests:

$$\min_{\pi} \; J(\pi) = \frac{1}{Q} \sum_{i \in \mathcal{Q}_\sqcup} L(u_i) + \eta \cdot P^{99} + \gamma \cdot \sum_{i \in \mathcal{Q}_t} \Pr(u_i = \text{EDGE} \mid r_i = \text{M}). \tag{4}$$

Here, P^{99} is the p99 tail latency, η controls the trade-off between average latency and tail latency, while γ represents the penalty for medium-risk requests that are not sent for cloud-side rechecking. This term captures the fact that medium-risk samples carry uncertainty and skipping rechecking reduces the security margin, while still allowing the system to maintain QoS under congestion by using a restricted response mode.

To ensure that high-risk requests are never allowed into the pass-through path, we use the following proxy security constraint:

$$\Pr(u_i = \text{EDGE} \mid r_i = \text{H}) \le \epsilon, \tag{5}$$

where ϵ is the allowed upper bound on the high-risk pass-through rate. In this paper, we set $\epsilon = 0$, meaning that high-risk requests are always blocked.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:44:13 UTC from IEEE Xplore. Restrictions apply.

To illustrate the computational complexity of the optimal dispatching policy, consider an offline version with a finite request set and a known set of medium-risk requests $\mathcal{M} = \{i : r_i = \mathrm{M}\}$, where low-risk requests are fixed to `EDGE` and high-risk requests are fixed to `BLOCK` ($\epsilon = 0$). For each medium-risk request, let a binary variable $y_i \in \{0, 1\}$ denote whether it is sent to the cloud for rechecking ($y_i = 1$) or downgraded to `EDGE` ($y_i = 0$). Suppose cloud-side rechecking consumes a budget a_i (equivalent resource cost) and yields a benefit b_i (equivalent reduction in security risk or rechecking utility). Under a total cloud budget B, the offline optimal dispatching problem can be written as

$$\max_{y_i \in \{0,1\}} \sum_{i \in \mathcal{M}} b_i y_i \quad \text{s.t.} \quad \sum_{i \in \mathcal{M}} a_i y_i \leq B. \tag{6}$$

This problem is isomorphic to the 0–1 knapsack problem. For any knapsack instance with item weights, values, and capacity W, one can construct a dispatching instance by setting $a_i = w_i$, $b_i = p_i$, and $B = W$, establishing a one-to-one correspondence between optimal solutions. Therefore, this offline constrained subproblem is NP-hard. The online case requires real-time decisions under unknown future arrivals and queue states and is no easier than the offline case. Hence, we adopt a computable online heuristic strategy for solution and evaluation.

D. Solution Strategy: Queue-Aware Three-Stage Gating

Based on the above formulation, we propose an online heuristic strategy called Queue-aware Three-stage Risk Gating (Q-TRiG). Its core idea is to keep the decision for high-risk requests deterministic while introducing queue-aware gating for medium-risk requests, thereby directly controlling ρ and W_q and suppressing tail-latency bursts under heavy load. Specifically, let t_i' denote the time when a request completes edge detection and arrives at the cloud. The cloud maintains the next available time $free_j(t_i')$ of each instance. The predicted waiting time is then

$$\hat{W}_i = \min_{1 \leq j \leq k} \max\{0, free_j(t_i') - t_i'\}. \tag{7}$$

We introduce a gating threshold θ. If $\hat{W}_i \leq \theta$, the request is sent for cloud-side rechecking; otherwise, it is downgraded to a restricted execution mode (e.g., disabling tool use, restricting context sources, or returning a safe template) before proceeding to the LLM. The only hyperparameter of Q-TRiG is θ. To avoid per-load post hoc tuning, we select a unified θ via grid search over representative load points and keep it fixed throughout the full-load evaluation.

Under Q-TRiG, high-risk requests are always mapped to `BLOCK`, thus satisfying the proxy security constraint. For medium-risk requests, the gating condition is $\hat{W}_i \leq \theta$. As θ increases, the set of requests satisfying this condition increases monotonically, and the medium-risk-to-cloud rechecking ratio increases monotonically as well. This makes θ an interpretable control knob that regulates the trade-off between rechecking intensity and queueing delay, and it can be combined with the downgrade penalty in the objective to achieve explicit balancing. The utilization of the cloud rechecking layer is approximately $\rho = \lambda p_{send}/(k\mu)$. When $\rho > 1$, the queue becomes unstable and waiting times diverge. Under congestion, Q-TRiG reduces p_{send} by switching some medium-risk requests from `CLOUD` to `EDGE`, thereby forming a feedback control on ρ and suppressing queue-dominated latency inflation in the heavy-load regime. The simulation results in Section IV further validate this mechanism.

IV. Experimental and Simulation Results

A. Simulation Setup

To verify the effectiveness of the proposed risk stratification and queue-aware gating strategy in improving tail latency under high load, we compare four strategies inspired by discrete-event queueing simulation and multi-server load-balancing models: Cloud-Only Rechecking (COR), Edge-Based Binary Gating (EBG), Static Three-Stage Gating (S-TRiG), and Queue-Aware Three-Stage Gating (Q-TRiG). The external arrival process is modeled as a Poisson process with arrival rate λ (req/s), scanned from 5 to 45. Each request first experiences edge detection, after which it follows one of three action paths according to the selected strategy:

1) `EDGE`: bypass cloud-side strong rechecking;
2) `CLOUD`: enter cloud-side rechecking;
3) `BLOCK`: directly blocked for high-risk requests.

To avoid bias from the initially empty system, the first 20% of requests are discarded as warm-up. For each (λ, π) pair, we run 10 simulation trials with different random seeds and report the average.

B. Results and Analysis

1) Cloud Waiting Time and Queue Stability: Fig. 2(a) shows the average cloud queueing delay as a function of λ. COR rapidly enters a divergent region after a relatively low load level, causing cloud waiting time to increase dramatically. The divergence point of EBG is shifted to the right, and S-TRiG further postpones that point. In contrast, Q-TRiG maintains extremely low cloud waiting time across the entire evaluated range, indicating that queue-aware gating effectively suppresses queue-dominated latency inflation at the cloud side.

2) p99 Tail Latency and Suppression in Heavy-Load Regimes: Fig. 2(b) presents the end-to-end p99 tail latency as a function of λ. Consistent with the trend of cloud waiting time, the three non-adaptive strategies exhibit rapid p99 growth beyond their respective critical points. By contrast, Q-TRiG keeps p99 at a low level throughout the entire range and shows no tail divergence even under high load, validating the strong capability of Q-TRiG to suppress tail latency in overloaded regimes.

3) Interpretation of Gating Behavior: Cloud-Offloading Ratio and Utilization Control: To explain the adaptive behavior of Q-TRiG, Fig. 2(c) illustrates the fraction of medium-risk requests sent to the cloud as λ increases. S-TRiG always sends 100% of medium-risk requests to the cloud, whereas Q-TRiG gradually reduces this fraction with increasing load,

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:44:13 UTC from IEEE Xplore. Restrictions apply.

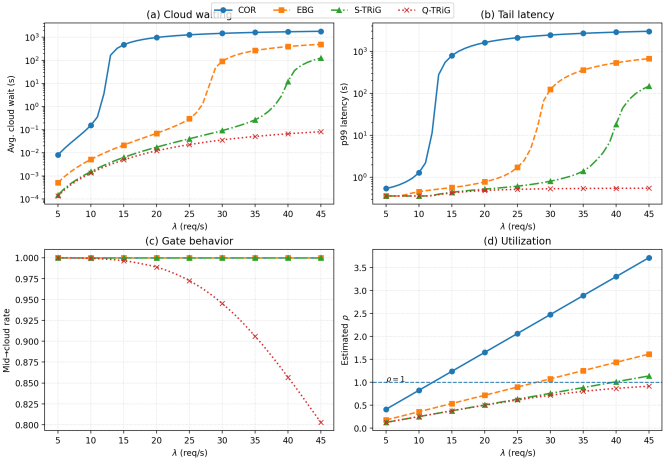


Fig. 2. Performance comparison of different defense strategies under increasing workload. (a) Average cloud queueing delay versus arrival rate. (b) End-to-end p99 tail latency versus arrival rate. (c) Fraction of medium-risk requests offloaded to the cloud versus arrival rate. (d) Estimated cloud utilization versus arrival rate.

thereby actively lowering the effective arrival rate into the cloud. Correspondingly, Fig. 2(d) shows the estimated cloud utilization:

$$\rho = \frac{\lambda p_{send}}{k\mu}. \tag{8}$$

The three non-adaptive strategies all experience $\rho > 1$ after their respective critical points, causing the queue to enter an unstable region. In contrast, Q-TRiG maintains $\rho < 1$ over the full range, which mechanistically explains why its cloud waiting time and p99 latency remain bounded.

V. Conclusion and Future Work

This paper investigates the system-level latency of prompt injection defense under a fixed cloud-computing budget and proposes an edge–cloud collaborative architecture with three-stage risk-aware decision. The edge performs lightweight risk stratification and selective offloading, while the cloud operates as a homogeneous multi-instance detection pool. By regulating the effective arrival rate to the cloud, the proposed design enables strict attribution of performance gains to edge-side dispatching. Simulation results show that under high utilization, the proposed three-stage mechanism significantly reduces queueing delay and postpones the p99 latency surge, thereby enlarging the stable operating region without increasing cloud capacity. Future work will explore risk-adaptive offloading and heterogeneous cloud detection to further improve efficiency under dynamic workloads.

References

[1] Y. Liang, C. Xu, Y. Mei, H. Zou, J. Guo, X. Fan, T. Wang, and H. Huang, "A comprehensive survey on large language model compression for artificial intelligence applications in edge systems," *IEEE Internet of Things Journal*, 2026.

[2] H. Zou, J. Guo, J. Zeng, Y. Li, J. Cao, and T. Wang, "Fine-grained service lifetime optimization for energy-constrained edge-edge collaboration," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 565–576.

[3] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, "Automatic and universal prompt injection attacks against large language models," *arXiv preprint arXiv:2403.04957*, 2024.

[4] Y. Liang, G. Li, G. Zhang, J. Guo, Q. Liu, J. Zheng, and T. Wang, "Latency reduction in immersive systems through request scheduling with digital twin networks in collaborative edge computing," *ACM Transactions on Sensor Networks*, 2024.

[5] H. Huang, T. Meng, J. Guo, X. Wei, and W. Jia, "Seceg: A secure and efficient strategy against ddos attacks in mobile edge computing," *ACM Transactions on Sensor Networks*, vol. 20, no. 3, pp. 1–21, 2024.

[6] Y. Liang, M. Yin, Y. Zhang, W. Wang, W. Jia, and T. Wang, "Grouping reduces energy cost in directionally rechargeable wireless vehicular and sensor networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 8, pp. 10 840–10 851, 2023.

[7] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 82 895–82 920, 2024.

[8] Y. Liang, G. Li, J. Guo, Q. Liu, X. Zheng, and T. Wang, "Efficient request scheduling in cross-regional edge collaboration via digital twin networks," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 2024, pp. 1–6.

[9] H. Huang, T. Meng, and W. Jia, "Joint optimization of prompt security and system performance in edge-cloud llm systems," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.

[10] T. Wang, Y. Liang, W. Jia, M. Arif, A. Liu, and M. Xie, "Coupling resource management based on fog computing in smart city systems," *Journal of Network and Computer Applications*, vol. 135, pp. 11–19, 2019.

[11] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.

[12] Y. Liang, M. Yin, W. Wang, Q. Liu, L. Wang, X. Zheng, and T. Wang, "Collaborative edge server placement for maximizing qos with distributed data cleaning," *IEEE Transactions on Services Computing*, vol. 18, no. 3, pp. 1321 – 1335, 2019.

[13] T. Wang, Y. Liang, X. Shen, X. Zheng, A. Mahmood, and Q. Z. Sheng, "Edge computing and sensor-cloud: Overview, solutions, and directions," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–37, 2023.

[14] H. Li and X. Liu, "Injecguard: Benchmarking and mitigating over-defense in prompt injection guardrail models," *arXiv preprint arXiv:2410.22770*, 2024.

[15] H. Zou, J. Guo, Y. Li, W. Fan, W. Su, C. Xu, Y. Liang, T. Wang, and J. Cao, "Logical correction enabled collaborative person detection inference in edge networks," *IEEE Transactions on Mobile Computing*, 2025.

[16] H. Li, R. Wen, S. Shi, N. Zhang, and C. Xiao, "Agentdyn: A dynamic open-ended benchmark for evaluating prompt injection attacks of real-world agent security system," *arXiv preprint arXiv:2602.03117*, 2026.

[17] Y. Liang, X. Bi, R. Shen, Z. He, Y. Wang, J. Xu, Y. Zhang, and X. Fan, "When mathematical methods meet artificial intelligence and mobile edge computing," *Mathematics*, vol. 13, no. 11, p. 1779, 2025.

[18] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[19] M. A. Ayub and S. Majumdar, "Embedding-based classifiers can detect prompt injection attacks," *arXiv preprint arXiv:2410.22284*, 2024.

[20] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "Promptshield: Deployable detection for prompt injection attacks," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, 2024, pp. 341–352.

[21] Y. Chen, H. Li, Y. Sui, Y. He, Y. Liu, Y. Song, and B. Hooi, "Can indirect prompt injection attacks be detected and removed?" in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 18 189–18 206.

[22] R. Xie, I. Grosof, and Z. Scully, "Heavy-traffic optimal size-and state-aware dispatching," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 1, pp. 1–36, 2024.

[23] Protect AI, "deberta-v3-base-prompt-injection-v2," https://hf-mirror.com/protectai/deberta-v3-base-prompt-injection-v2, 2024, hugging Face model page, accessed: 2026-04-08.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:44:13 UTC from IEEE Xplore. Restrictions apply.